Guida al sistema

Come è fatta la casa editrice del repository ZeusBot: chi fa cosa, il percorso di un libro, le regole.

Per Felice, autore. Ottobre 2026.

Fotografia del ramo `motore` al commit `f7decc4`.

> **Come leggere.** Tra parentesi, in grigio, c'è il file da cui viene ogni affermazione. Quando una cosa non c'è nel repository, la guida lo scrive: «non esiste».

Indice

0. Le parole da sapere 3
1. A cosa serve il sistema 5
2. Organigramma: chi fa cosa 7
3. Il percorso di un libro 18
4. Le regole 30
5. I rami 38
6. Cosa non esiste 42
7. Appendici e indice 45
8. I 27 punti deboli 49
9. Dove trovare le cose 55

0. Le parole da sapere

Poche parole tecniche tornano spesso. Eccole, una volta per tutte.

- **Repository**: l'archivio del progetto su GitHub, con tutta la storia delle modifiche. Qui si chiama ZeusBot.
- **Ramo**: una versione parallela del repository. Ogni ramo ha i suoi file. Il ramo `motore` contiene la macchina; ogni libro vive su un ramo suo.
- **Commit**: un salvataggio con nome e data. Ha un codice breve, per esempio `f7decc4`.
- **Push**: l'invio dei salvataggi su GitHub (su «origin», la copia in rete). Finché non c'è il push, il lavoro esiste solo nel computer di lavoro.
- **Merge**: l'unione di un ramo dentro un altro.
- **Script**: un piccolo programma che fa un lavoro preciso (contare parole, fare il PDF…).
- **Hook**: un guardiano automatico. Claude Code lo chiama prima di certe

azioni; l'hook può lasciar passare, avvisare o bloccare.

- **Claude Code**: l'assistente che lavora nel repository, legge le regole e lancia i comandi.
- **YAML**: un formato di testo per i dati, fatto di righe «nome: valore». I file finiscono in `.yaml`.
- **Gate**: una fermata obbligata. Il lavoro si ferma finché l'autore non approva.
- **sha256**: l'«impronta digitale» di un file. Se cambia anche una virgola, l'impronta cambia.
- **KDP**: Kindle Direct Publishing, il servizio di Amazon per pubblicare.
- **Metadati del PDF**: dati nascosti nel file, come Title (titolo) e Author (autore).
- **Slug**: il nome breve di un libro usato nei percorsi, senza spazi né accenti.

1. A cosa serve il sistema

Il sistema è una fabbrica di libri. Serve per romanzi di qualsiasi genere (motore/README.md).

Il cuore è il **motore**: procedura, script, modelli vuoti, profili di genere e dati KDP. Il motore non contiene libri veri (motore/README.md).

Il percorso è sempre lo stesso. L'autore scrive un briefing. Il motore crea la cartella del libro. Claude Code scrive i documenti e poi i capitoli, uno alla volta. A ogni tappa importante l'autore approva. Alla fine il motore controlla il testo, compone il libro, fa il PDF di stampa e prepara il pacchetto per KDP (motore/README.md, «Dal briefing al PDF»).

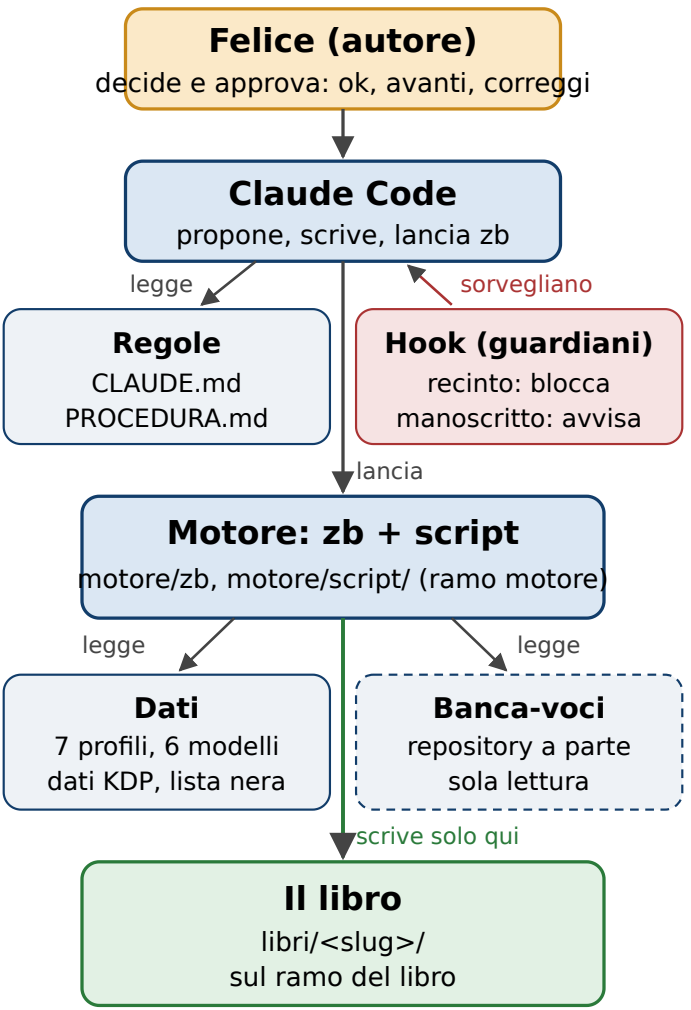
Tre idee reggono tutto (CLAUDE.md):

1. **Il motore resta pulito.** Il ramo `motore` non contiene mai un libro.

2. **Si scrive solo nella cartella del libro.** Mai nel motore, mai altrove.
3. **Proponi, l'autore approva, poi applica.** Claude Code non approva mai da solo.

2. Organigramma: chi fa cosa

Lo schema



Schema ricavato da CLAUDE.md, motore/README.md e docs/INFRASTRUTTURA_COMPLETA.md §3 e §8.4.

Chi fa cosa, in breve

Chi	Che cosa fa	Dove sta
Felice	Scrive il briefing, risponde ok / avanti / correggi, decide	—
Claude Code	Legge le regole, propone, scrive nel libro, lancia `zb`	—
Motore	Crea, controlla, compone, impagina	`motore/`

Chi	Che cosa fa	Dove sta
Regole	Dicono come si lavora	`CLAUDE.md`, `motore/PROCEDURA.md`
Hook	Sorvegliano le scritture	`motore/script/hook_*.py`
Libro	Il lavoro vero	`libri/<slug>/`

(CLAUDE.md; motore/README.md; docs/INFRASTRUTTURA_COMPLETA.md §8.4)

Il motore: `zb` e gli script

`zb` è il comando unico. Si scrive `python3 motore/zb <comando> <libro>`. Il libro si indica sempre con il percorso della sua cartella (motore/zb; motore/README.md, «Comandi»).

Ogni comando di `zb` lancia uno script della cartella `motore/script/`. Gli script sono 28 file (motore/zb, tabella SCRIPT; cartella motore/script/).

I comandi principali (motore/README.md, «Comandi»):

- `zb nuovo`: crea il libro dal briefing.
- `zb avvio`: controlli di inizio sessione e riga «Letto: …».
- `zb stato`: mostra a che punto è il libro. Non scrive nulla.
- `zb pronto`: chiude il lavoro di una fase e apre la fermata.
- `zb ok`, `zb avanti`, `zb correggi`: le risposte dell'autore.
- `zb esito`: controlla un capitolo appena scritto.
- `zb continuita`, `zb ortografia`, `zb compila`, `zb impagina`, `zb pdf`, `zb pacchetto`, `zb kdp`: la chiusura.
- `zb prove`: le prove del motore sui mini-libri inventati.

Nessun comando del motore fa commit o push da solo (motore/PROCEDURA.md §8).

La «Bibbia editor»

Nel repository «Bibbia editor» è il nome dato all'insieme delle regole editoriali valide per ogni libro. Il commit `668afc2` si chiama proprio «Bibbia editor: appendici e indice» e modifica `motore/PROCEDURA.md` (docs/INFRASTRUTTURA_COMPLETA.md §4).

Le regole stanno in due file (docs/INFRASTRUTTURA_COMPLETA.md §4):

- `CLAUDE.md`: le regole di avvio. Claude Code le legge a ogni sessione.
- `motore/PROCEDURA.md`: la procedura completa, in 11 sezioni.

Le aiutano alcuni file di dati, per esempio la lista nera di base (`motore/dati/lista-nera.yaml`) e le parole italiane ammesse (`motore/dati/parole-ammesse-it.txt`) (docs/INFRASTRUTTURA_COMPLETA.md §4).

> Attenzione: la `bibbia.md` di ogni libro (personaggi, luoghi, fatti) è un'altra cosa. Sta nella cartella del libro, in `02-bibbia/` (docs/INFRASTRUTTURA_COMPLETA.md §4).

Profili di genere

Un profilo dà i valori predefiniti di un genere: lunghezza, capitoli, scene, frase media, dialogo (motore/PROCEDURA.md §3).

Sono 7, nella cartella `motore/profili/`: fantascienza, fantasy, giallo, narrativa letteraria, romance, thriller, young adult (cartella motore/profili/).

Un valore del briefing fuori dal profilo è accettato solo con un «Motivo override:» scritto nel briefing. Senza motivo, `zb nuovo` si ferma (motore/PROCEDURA.md §3).

Voci narranti

Una voce dice **come** si racconta. Il **cosa** lo decide l'autore nel briefing (motore/README.md, «Banca-voci»).

Le voci stanno in un repository separato, la banca-voci, mai dentro il motore. Il motore le legge in sola lettura dalla cartella indicata da `ZB_BANCA_VOCI` (motore/README.md, «Banca-voci»).

Oggi le voci sono 11, tutte pronte. L'elenco è in `indice.yaml` della banca-voci e in `motore/passaggio-di-consegne.md`, «Stato» (motore/passaggio-di-consegne.md). Questa guida non le nomina.

Ogni voce ha una scheda in 7 sezioni e un elenco di espressioni vietate, `divieti.yaml` (motore/README.md, «Banca-voci»).

Un libro sceglie la voce nel briefing («Voce dalla banca:», «Ambientazione:»); `zb nuovo` la scrive in `libro.yaml` (motore/README.md, «Banca-voci»).

Se il briefing contrasta con la voce, vince il briefing, e il contrasto si segnala (motore/modelli/briefing.md; README della banca-voci, copiato in docs/INFRASTRUTTURA_COMPLETA.md §6).

Modelli

I modelli sono i file vuoti da cui nasce un libro. Stanno in `motore/modelli/` e sono 6 (cartella motore/modelli/):

File	A cosa serve
`briefing.md`	Il modulo che compila l'autore
`libro.yaml`	Le regole del libro
`stato.yaml`	A che punto è il libro
`cronologia.yaml`	Date ed eventi
`LEGGIMI.md`	Cosa leggere a ogni avvio
`conferme-autore.yaml`	Le conferme per KDP

(motore/modelli/; docs/INFRASTRUTTURA_COMPLETA.md §3.3)

Hook

Gli hook sono tre. Stanno nel modello `motore/dati/hook-settings.esempio.json` (motore/dati/hook-settings.esempio.json):

Hook	Quando	Cosa fa
recinto	prima di ogni scrittura di file	**Blocca** le scritture fuori dal libro attivo
manoscritto	prima di scrivere in `04-manoscritto/`	Oggi **avvisa** se manca `zb avvio`
sessione	all'inizio della sessione	Annota la sessione

(motore/script/hook_recinto.py; motore/script/hook_manoscritto.py; motore/dati/hook.yaml; motore/PROCEDURA.md §10)

Gli hook si accendono solo sui rami dei libri, con `zb hook attiva --ok`. Sul ramo `motore` sono spenti per scelta (motore/passaggio-di-consegne.md, «Stato»). Dettagli nel capitolo 4.

CLAUDE.md

È il foglio delle regole di avvio. Sta nella radice del repository e Claude Code lo legge a ogni sessione (docs/INFRASTRUTTURA_COMPLETA.md §4 e §9).

Ha al massimo 30 righe e nessun nome di libro: lo controlla una prova del motore (docs/INFRASTRUTTURA_COMPLETA.md §9; motore/script/prove.py). Cosa dice: capitolo 4.

Rami dei libri

Ogni libro vive su un ramo suo, nella cartella `libri/<slug>/`. Quel ramo porta anche una copia del motore (docs/INFRASTRUTTURA_COMPLETA.md §8.3). Dettagli nel capitolo 5.

3. Il percorso di un libro

Dall'idea a KDP ci sono quattro tratti: nascita, documenti, stesura, chiusura (motore/PROCEDURA.md §3-7).

Tratto 1. La nascita

1. Sul ramo del libro, mai sul ramo `motore`, l'autore compila il modello `motore/modelli/briefing.md` e lo mette in `libri/<slug>/briefing.md` (docs/INFRASTRUTTURA_COMPLETA.md §3.3).
2. Claude Code lancia `zb nuovo libri/<slug>/briefing.md` (CLAUDE.md; motore/PROCEDURA.md §3).
3. Se il briefing è incompleto, `zb nuovo` elenca al massimo 5 cose mancanti e **non crea nulla** (motore/PROCEDURA.md §3).
4. Se è completo, crea la cartella con `libro.yaml`, `stato.yaml`, `LEGGIMI.md`, `cronologia.yaml`, `nomi_propri.txt`, la bibbia, il manuale di stile, la scaletta e

il piano parole (motore/PROCEDURA.md §3).

5. Scrive `libri/libro-attivo.md`, così il recinto sa qual è il libro attivo (docs/INFRASTRUTTURA_COMPLETA.md §3.3).

Per un libro che ha già dei documenti, ma non `libro.yaml`, c'è `zb adotta`: prepara delle bozze con la fonte di ogni valore (motore/PROCEDURA.md §3b).

Prima di ogni sessione: `zb avvio`

A ogni sessione, e dopo ogni compattazione della memoria di Claude Code, si lancia `zb avvio <libro>` (motore/PROCEDURA.md §4).

Si ferma senza scrivere nulla se qualcosa non va. Per esempio: modifiche non salvate, ramo sbagliato, ramo non allineato a GitHub, un documento approvato cambiato di nascosto (motore/PROCEDURA.md §4).

Se tutto va bene stampa la riga «Letto: …», con i file letti, le righe e le impronte.

Claude Code deve riportarla prima di scrivere (motore/PROCEDURA.md §4; CLAUDE.md).

Tratto 2. Documenti e pagina campione

Fase	Lavoro	Fermata
documenti	bibbia, cronologia, scaletta, piano parole	gate «documenti»
pagina campione	`05-revisioni/pagina-campione.md`	gate «pagina_campione»

(motore/PROCEDURA.md §5)

Quando una fase è finita, Claude Code lancia `zb pronto`. Il motore apre la fermata e mostra il primo blocco di circa 120 righe (motore/PROCEDURA.md §5).

Le risposte dell'autore (motore/PROCEDURA.md §5):

- **avanti**: il blocco dopo. Non approva nulla.
- **ok**: approva. Vale solo se l'autore ha visto tutti i blocchi.
- **ok, lotti da N**: approva e fissa quanti capitoli scrivere di fila.
- **correggi: …**: registra una correzione. La fermata resta aperta.
- **stato**: mostra lo stato, senza cambiare nulla.

Qualunque altra frase è una domanda: non cambia lo stato (motore/PROCEDURA.md §5).

Tratto 3. La stesura

Per ogni capitolo (motore/PROCEDURA.md §6):

1. Claude Code rilegge LEGGIMI, manuale, direttive, scaletta, piano, cronologia e capitolo precedente.
2. Riporta la riga «Letto».
3. Scrive `04-manoscritto/NN-<slug>.md`.
4. Lancia `zb esito <libro> N`: parole, stile, continuità, ripetizioni. Il report va

   in `06-diagnostica/capitoli/`.
5. Salva: commit e push.

Se un controllo fallisce: **una sola** correzione mirata, poi di nuovo `zb esito`. Al secondo fallimento di fila ci si ferma e decide l'autore (motore/PROCEDURA.md §6).

Dopo il capitolo 3 c'è la fermata «primi_capitoli». Con l'ok l'autore sceglie i lotti; poi i capitoli si scrivono di fila (motore/PROCEDURA.md §6).

Scritto l'ultimo capitolo del piano parole, la fase passa da sola a «chiusura» (motore/PROCEDURA.md §5-6).

Tratto 4. La chiusura

L'ordine è fisso (motore/PROCEDURA.md §7):

1. continuita
2. ortografia
3. compila
4. impagina

5. pdf
6. pacchetto
7. kdp
8. pronto

`pacchetto` va prima di `kdp`: il controllo KDP legge la scheda del pacchetto (motore/PROCEDURA.md §7).

Nelle pagine seguenti, un passo per pagina. «`<titolo>`» è il titolo del libro trasformato in nome di file.

Passo 1. `zb continuita`

- **Cosa fa.** Controlla date, giorni della settimana, età dei personaggi, durate, cifre, nomi scritti in due modi, ordine delle date.
- **Cosa serve prima.** I capitoli in `04-manoscritto/` e il file `cronologia.yaml` del libro.
- **Cosa produce.** Il report `06-diagnostica/continuita.md`, con una lista di controllo per capitolo.

(motore/script/continuita.py; motore/PROCEDURA.md §7)

Passo 2. `zb ortografia`

- **Cosa fa.** Tre livelli: (a) Hunspell, un dizionario italiano automatico, sempre; (b) LanguageTool, un correttore più fine, solo se installato; (c) una lista di controllo per la lettura umana, sempre.
- **Cosa serve prima.** Hunspell con il dizionario italiano; le parole ammesse del libro (`nomi_propri.txt`). Per il livello (b), LanguageTool e la variabile `ZB_LANGUAGETOOL`.
- **Cosa produce.** Il report `06-diagnostica/controllo-ortografico.md`. Dice quali livelli sono stati eseguiti.

(motore/script/ortografia.py; motore/PROCEDURA.md §7)

Passo 3. `zb compila`

- **Cosa fa.** Mette insieme il libro: frontespizio, pagina del copyright, sommario, parti, capitoli in ordine, appendici. Il sommario lo genera sempre il motore.

- **Cosa serve prima.** `libro.yaml` e i capitoli. Le appendici dichiarate in `libro.yaml` e presenti in `05-appendici/`.
- **Cosa produce.** `05-output/<titolo>-completo.md`. Le immagini delle appendici vanno in `05-output/appendici/`.

(motore/script/compila.py; motore/PROCEDURA.md §7)

Passo 4. `zb impagina`

- **Cosa fa.** Fa il PDF di stampa con Chromium (un navigatore web usato come tipografo). Margini dalla tabella KDP, pagine iniziali senza numero, sommario con le pagine vere, formato esatto della pagina, metadati Title e Author.
- **Cosa serve prima.** Il file compilato del passo 3; nel computer di lavoro Chromium, PyMuPDF e markdown.
- **Cosa produce.** `05-output/<titolo>.pdf`.

(motore/script/impagina.py; motore/README.md, «Ambiente»)

Passo 5. `zb pdf`

- **Cosa fa.** Verifica il PDF: numero di pagine; Title e Author uguali a `libro.yaml`; font incorporati; margini contro i minimi KDP; ogni voce del sommario sulla pagina giusta.
- **Cosa serve prima.** Il PDF del passo 4; gli strumenti `pdfinfo`, `pdffonts`, `pdftotext`.
- **Cosa produce.** Il report `06-diagnostica/verifica-pdf.md`.

(motore/script/verifica_pdf.py)

Passo 6. `zb pacchetto`

- **Cosa fa.** Prepara la cartella per KDP con modelli già compilati con i dati di `libro.yaml`. **Non** scrive testi commerciali (descrizione, quarta, parole chiave, categorie): li scrive Claude Code dopo. Non sovrascrive mai un file che c'è già.
- **Cosa serve prima.** `libro.yaml`.

- **Cosa produce.** In `06-pubblicazione/`: `scheda-amazon.md`, `quarta.md`, `brief-copertina.md`, `conferme-autore.yaml`, `promemoria-ai.md`, `checklist.md`.

(motore/script/pacchetto.py)

Passo 7. `zb kdp`

- **Cosa fa.** Controlla la conformità a KDP in 15 sezioni: titolo, sottotitolo, serie, descrizione, autore, editore, parole chiave, categorie, copertina, ISBN, cartaceo, eBook, dichiarazione sull'IA, contenuti, recensioni. Ciò che deve confermare l'autore, finché non è confermato, risulta KO.
- **Cosa serve prima.** `libro.yaml`, la scheda del passo 6, il file compilato, il PDF, `conferme-autore.yaml`, i dati KDP del motore; l'eBook `ebook.docx` se il libro lo prevede.
- **Cosa produce.** `06-pubblicazione/conformita-kdp.md`, una riga OK/KO per sezione.

(motore/script/conformita_kdp.py; docs/INFRASTRUTTURA_COMPLETA.md

§3.6)

Passo 8. `zb pronto`

- **Cosa fa.** Controlla che ci siano tutti i risultati dei passi 1-7. Se sì, il libro passa a «chiuso».
- **Cosa serve prima.** I sette file: `continuita.md`, `controllo-ortografico.md`, il compilato, il PDF, `verifica-pdf.md`, `checklist.md`, `conformita-kdp.md`.
- **Cosa produce.** Il cambio di fase in `stato.yaml`. Se manca qualcosa, elenca cosa e con quale comando farlo.

(motore/script/fase.py, elenco CHIUSURA e cmd_pronto)

> **Prima di pubblicare.** I dati KDP del motore non sono mai stati riverificati dall'autore sulle pagine ufficiali (`ultima_verifica: null`). La verifica si registra con `zb verifica KDP fatta` (motore/report-motore.md §11;

> motore/passaggio-di-consegne.md, «Punti aperti» 1).

Dopo ogni passo: il salvataggio

Dopo ogni passo che scrive: report in `06-diagnostica/`, commit, push, controllo dello stato. Se il push fallisce, ci si ferma e si riporta l'errore. Mai dire «fatto» se il lavoro non è su GitHub (CLAUDE.md; motore/PROCEDURA.md §8).

4. Le regole

Cosa dice CLAUDE.md

Il file ha 29 righe (docs/INFRASTRUTTURA_COMPLETA.md §4). In parole semplici dice (CLAUDE.md):

1. **Regola madre.** Il motore resta pulito; il ramo `motore` non contiene mai un libro.
2. **Prima azione per un briefing nuovo:** `zb nuovo`, che crea la cartella del libro. Da lì si scrive solo nella cartella del libro.
3. **Recinto.** L'hook blocca le scritture fuori dal libro attivo. Solo se l'autore scrive «lavoro sul motore», Claude Code lancia `zb lavoro motore`.
4. **Letture.** Per lavorare su un libro: `motore/PROCEDURA.md` e il `LEGGIMI.md` del libro. Se manca il percorso del libro, Claude Code lo chiede prima di toccare file.
5. **Avvio.** Prima di scrivere: `zb avvio <libro>` e la riga «Letto». Se l'avvio si

ferma, Claude Code si ferma e dice perché.

6. **Prima di ogni capitolo:** di nuovo `zb avvio`, la scheda della voce (se c'è) e il briefing.
7. **Cosa legge il motore:** solo `motore/`, la cartella del libro e la banca-voci (in sola lettura).
8. **Report di sessione** ogni 5 scambi, o quando l'autore scrive «passaggio». Va nella cartella del libro (`passaggio-di-consegne.md`). Sette sezioni fisse: Stato; Decisioni prese; Regole da non violare; Fatto in questa sessione; Punti aperti; Prossimo passo; Ripartenza.
9. **Precedenza** delle regole (vedi più avanti).
0. **Metodo:** proponi, l'autore approva, applica. Mai approvare da solo.
1. **Risposte dell'autore:** ok, avanti, correggi, stato.
2. **Ramo:** quello della sessione, letto da git.
3. **Tono:** il testo precedente è un registro dei fatti. Non si giudica.

4. **Salvataggio** a ogni passo: commit, push, controllo. Resoconto con codice del commit, file e «branch allineato a origin: sì/no».

Gli hook

Il recinto: l'unico che blocca

- **Cosa guarda.** Ogni volta che Claude Code vuole scrivere un file (con gli strumenti Write, Edit, MultiEdit, NotebookEdit).
- **Regola.** Si può scrivere solo in `libri/<slug-attivo>/`. Tutto il resto, `motore/` compreso, è rifiutato con il messaggio «RECINTO: …». Lo strumento non parte.
- **Libro attivo.** È la prima riga di `libri/libro-attivo.md`. Lo scrivono `zb nuovo` e `zb lavoro libro <slug>`.
- **Nascita.** È permesso anche scrivere il briefing di un libro che non ha ancora `libro.yaml`.
- **Lavoro sul motore.** Quando l'autore scrive «lavoro sul motore», Claude Code lancia `zb lavoro motore`: allora

si può scrivere nel repository, tranne `libri/`.
- **Senza libro attivo** (succede sul ramo `motore`): solo un avviso, nessun blocco.

(motore/script/hook_recinto.py; docs/INFRASTRUTTURA_COMPLETA.md §3.2)

Limiti dichiarati. Il recinto non controlla i comandi di shell (Bash): una scrittura fatta così passa. E non può sapere chi ha chiesto «lavoro sul motore» (motore/passaggio-di-consegne.md, «Punti aperti» 8 e 9).

Il manoscritto: avvisa

Se si scrive in `04-manoscritto/` senza aver fatto `zb avvio` in quella sessione, oggi stampa «ATTENZIONE: …» e lascia passare. Lo annota in `.zb/avvisi-hook.log` del libro (motore/PROCEDURA.md §10).

La modalità sta in `motore/dati/hook.yaml`: oggi `avviso`. Il

passaggio a `blocco` sarà un commit a parte, dopo l'ok dell'autore (motore/dati/hook.yaml; motore/PROCEDURA.md §10).

La sessione: prende nota

All'inizio di ogni sessione scrive il codice della sessione. Dopo una compattazione della memoria, i vecchi «avvio fatto» non valgono più (motore/PROCEDURA.md §10).

Come si accendono

`zb hook attiva --ok`, solo sui rami dei libri e solo dopo l'ok dell'autore. Crea `.claude/settings.json`. `zb hook stato` dice se sono accesi; `zb hook disattiva` li spegne (motore/PROCEDURA.md §10; docs/INFRASTRUTTURA_COMPLETA.md §3.2).

Gli hook sono «una protezione in più, non una garanzia» (motore/PROCEDURA.md §10, titolo). I controlli che non dipendono dagli hook li fa `zb avvio`, a ogni sessione.

Altre protezioni, senza hook

Chi	Cosa fa
Codice del motore	Ogni comando scrive solo nella cartella del libro
`zb nuovo`	Rifiuta un libro sul ramo `motore` o fuori da `libri/<slug>/`
`zb recinto`	Prova che i comandi non scrivano fuori dal libro
`zb separazione`	Prova che in `motore/` non ci siano pezzi di libri veri

(docs/INFRASTRUTTURA_COMPLETA.md §3.1)

L'ordine di precedenza

Quando due regole si contraddicono, vince quella più in alto. Il repository ne dà due versioni.

In `motore/PROCEDURA.md` §1:

1. manuale del libro e `libro.yaml`
2. profilo di genere
3. la procedura del motore
4. skill dell'autore

In `CLAUDE.md`:

1. manuale del libro (e `libro.yaml`)
2. procedura del motore
3. skill dell'autore

In tutti e due i casi: **ogni contrasto si segnala** (motore/PROCEDURA.md §1; CLAUDE.md).

La differenza: `CLAUDE.md` non nomina il profilo di genere. Ma le due scale non si contraddicono: il profilo sta sotto il libro e sopra la procedura (docs/INFRASTRUTTURA_COMPLETA.md §10, punto 11).

C'è poi una regola a parte per la voce: se il briefing contrasta con la scheda della voce, vince il briefing (motore/modelli/briefing.md). Nessuna

delle due scale dice dove stanno voce e briefing (docs/INFRASTRUTTURA_COMPLETA.md §10, punto 11).

Le «skill dell'autore» sono nominate, ma nel repository non esistono (capitolo 6).

5. I rami

Il ramo `motore`

- Contiene solo `motore/`, `CLAUDE.md`, `.gitignore` e la cartella `docs/` con i documenti di lettura.
- Mai un libro, mai una cartella `libri/`.
- È l'unico ramo dove si lavora sul motore.
- Qui gli hook sono spenti: non c'è `.claude/settings.json`, e una prova lo richiede.
- Qui `zb nuovo` e `zb lavoro libro` si rifiutano.

(docs/INFRASTRUTTURA_COMPLETA.md §8.3; motore/passaggio-di-consegne.md, «Stato» e «Decisioni prese»; contenuto del ramo al commit f7decc4)

I rami dei libri

- Un libro per ramo, nella cartella `libri/<slug>/`, con `libri/libro-attivo.md`.

- Ogni ramo porta una copia di `motore/`. Il motore aggiornato arriva con un merge dal ramo `motore`, mai con un rebase.
- Con gli hook accesi c'è anche `.claude/settings.json`.

(docs/INFRASTRUTTURA_COMPLETA.md §8.3; motore/passaggio-di-consegne.md, «Prossimo passo»)

Al momento della scansione, su GitHub c'erano 5 rami con una cartella `libri/`. Uno solo aveva il motore aggiornato e gli hook accesi. Gli altri 4 hanno un motore più vecchio, o nessun motore, e un `CLAUDE.md` diverso. Uno di questi usa modelli propri in `libri/_modelli/`, un sistema di prima del motore (docs/INFRASTRUTTURA_COMPLETA.md §8.3 e §10, punto 17).

I nomi dei rami dei libri non si scrivono sul ramo `motore`: lo vieta la regola sui nomi reali (motore/README.md, «Solo il motore nel repository»).

I libri vecchi: una scelta, non un errore

I rami dei libri vecchi sono **copie di lavoro per la riscrittura**. Hanno un `CLAUDE.md` proprio **per scelta**, e non vanno riallineati al motore.

> Fonte di questa scelta: la decisione dell'autore, comunicata nella richiesta di questa guida. Nel repository non è scritta in nessun file. L'archivio `docs/INFRASTRUTTURA_COMPLETA.md` la elenca tra i punti deboli (§10, punto 17); qui va letta come scelta dichiarata.

Cosa comporta, in pratica:

- Le regole di questa guida valgono pienamente solo sul ramo `motore` e sul ramo del libro aggiornato (docs/INFRASTRUTTURA_COMPLETA.md §10, punto 17).
- Su un ramo vecchio comanda il suo `CLAUDE.md`.

- Un aggiornamento del motore non arriva a un ramo di libro finché non si fa il merge, ramo per ramo (docs/INFRASTRUTTURA_COMPLETA.md §10, punto 18).

Altri rami

Ramo	Cosa contiene
`archivio/ghimoney`	Un progetto separato tolto dal motore. Solo archivio
`ccr-*`, `claude/*`	Lavori di sessione non legati al motore, o libri di vecchio formato

(docs/INFRASTRUTTURA_COMPLETA.md §8.3)

6. Cosa non esiste

Alcune cose sono nominate o attese, ma nel repository non ci sono.

Skill e comandi slash: non esistono

Non c'è `.claude/skills/`, non c'è `.claude/commands/`, non c'è un `SKILL.md` del sistema, su nessun ramo del sistema (docs/INFRASTRUTTURA_COMPLETA.md §7 e §10, punto 3).

`CLAUDE.md` e `PROCEDURA.md` citano le «skill dell'autore» nella precedenza, ma il repository non ne contiene. I «comandi» del sistema sono i comandi `zb` (docs/INFRASTRUTTURA_COMPLETA.md §7).

Sezione anti-scrittura-da-IA: non esiste

Nessuna regola chiede di alternare frasi lunghe e corte per evitare un testo «da IA». Nessuna regola chiede «imperfezioni

naturali» (docs/INFRASTRUTTURA_COMPLETA.md §4 e §10, punto 1).

Esistono solo strumenti vicini (docs/INFRASTRUTTURA_COMPLETA.md §4):

- frase media e percentuale di dialogo per genere, nei profili, controllate da `stile.py`;
- la lista nera di base, `motore/dati/lista-nera.yaml`;
- i tetti e le parole vietate del libro;
- i divieti e le regole di ritmo delle singole voci.

Correttore professionale per l'italiano: non esiste

Non c'è un documento o uno strumento con questo nome (docs/INFRASTRUTTURA_COMPLETA.md §4 e §10, punto 2).

Esiste solo il **controllo ortografico a tre livelli** di `zb ortografia`: Hunspell,

LanguageTool se installato, lista per la lettura umana (motore/PROCEDURA.md §7).

LanguageTool non è installato di default. Senza la variabile `ZB_LANGUAGETOOL` il secondo livello salta (docs/INFRASTRUTTURA_COMPLETA.md §10, punto 2).

Stile, «e» al posto di «è», accordi sbagliati e parole vere usate al posto di altre restano alla lettura umana (motore/PROCEDURA.md §7).

Anche questo non esiste

- Un file con l'elenco delle dipendenze (`requirements.txt` o simili). Stanno solo scritte in `motore/README.md` (docs/INFRASTRUTTURA_COMPLETA.md §8.1).
- Un controllo automatico che lanci le prove a ogni modifica: le prove si lanciano a mano (docs/INFRASTRUTTURA_COMPLETA.md §7).

7. Appendici e indice

Come stanno oggi

> Oggi il motore **include** le appendici. Lo fa dal commit `71a36cc` («Motore: appendici dichiarate in libro.yaml e sommario completo», 14 ottobre 2026). Le regole sono in `motore/PROCEDURA.md` §7 dal commit `668afc2` (storia git del ramo motore; motore/script/compila.py).

Appendici. Sono i materiali dopo l'ultima unità del libro, epilogo compreso: soluzioni di enigmi, mappe, glossari, note (motore/PROCEDURA.md §7).

- Stanno nella cartella `05-appendici/` del libro (motore/dati/struttura-libro.yaml).
- Si dichiarano in `libro.yaml`, campo `appendici`, nell'ordine di lettura: titolo, file, didascalia per le immagini (motore/PROCEDURA.md §7; motore/dati/libro.schema.yaml).

- Formati: testo `.md`; immagini `.svg`, `.png`, `.jpg`. Un `.svg` diventa PNG con Chromium (motore/PROCEDURA.md §7).
- Ogni appendice comincia su una pagina nuova (motore/PROCEDURA.md §7).
- Un file dichiarato e assente ferma `zb compila`. Un file presente ma non dichiarato dà solo un avviso e resta fuori (motore/script/compila.py).
- Le appendici non vanno mai in `04-manoscritto/` (motore/PROCEDURA.md §7).
- Un'appendice non anticipa nulla: le soluzioni stanno solo in appendice (motore/PROCEDURA.md §7).
- Un libro senza il campo `appendici` si compila come prima (motore/PROCEDURA.md §7).

Indice (sommario). Lo genera `zb compila`, sempre. Non si scrive a mano (motore/PROCEDURA.md §7).

- Contiene, in ordine: parti, prologo, capitoli («N. Titolo»), interludi, epilogo, appendici (motore/PROCEDURA.md §7).
- `zb impagina` aggiunge la pagina vera di ogni voce; `zb pdf` controlla che sia giusta (motore/PROCEDURA.md §7).
- Per cambiare una voce si cambia la sua fonte (titolo del capitolo, `libro.yaml`), poi si rilancia `zb compila` (motore/PROCEDURA.md §7).

Cosa è stato deciso

Decisione	Stato nel repository
Cartella appendici nel libro	Fatto: `05-appendici/`
Indice sempre presente	Fatto: lo genera `zb compila`
Motore generico	Fatto: le regole valgono per ogni libro; nessun libro nel motore

(motore/dati/struttura-libro.yaml; motore/PROCEDURA.md §7; motore/README.md; CLAUDE.md)

Resta un dettaglio. Il modello `motore/modelli/libro.yaml` non mostra il campo `appendici`, anche se lo schema lo prevede (docs/INFRASTRUTTURA_COMPLETA.md §10, punto 15).

8. I 27 punti deboli

Sono i punti della sezione 10 di `docs/INFRASTRUTTURA_COMPLETA.md`, riscritti in parole semplici. Sono ordinati per importanza, dal più serio al più lieve. Tra parentesi quadre il numero originale. L'ordine è una valutazione di questa guida, non del repository.

Importanti: possono lasciar passare un errore

1. Il recinto non vede la shell. Una scrittura fatta con un comando di shell (Bash) fuori dal libro non viene bloccata [6].

2. Sul ramo `motore` non c'è recinto. Lì gli hook sono spenti per scelta. Restano la prova di separazione e la disciplina [8].

3. «Lavoro sul motore»: nessuno verifica chi lo dice. Claude Code lancia `zb lavoro motore` quando legge la frase. L'hook non può sapere se l'ha scritta davvero l'autore [7].

4. Gli aggiornamenti del motore non arrivano da soli ai libri. Ogni ramo di libro ha la sua copia del motore. Serve un merge, ramo per ramo [18].

5. Le voci copiate sul ramo `motore`. La banca-voci dice che le voci non vanno mai nel repository del motore. L'archivio `INFRASTRUTTURA_COMPLETA.md` le copia per intero in `docs/`, su richiesta esplicita. Il controllo di separazione guarda solo `motore/` e non se ne accorge [24].

6. Nomi reali sul ramo `motore`. Le schede delle voci copiate nell'archivio nominano autori reali. Il motore vieta i nomi reali; l'archivio sta fuori da `motore/`, ma sullo stesso ramo [25].

7. Il report ogni 5 scambi non lo controlla nessuno. È solo una regola scritta in `CLAUDE.md` [5].

8. Due scale di precedenza. `PROCEDURA.md` mette il profilo di genere; `CLAUDE.md` no. E nessuna dice dove stanno voce e briefing [11].

9. Nessuna regola anti-scrittura-da-IA. Ci sono solo strumenti vicini (capitolo 6) [1].

10. Nessun correttore professionale. C'è il controllo a tre livelli; LanguageTool non è installato di default [2].

11. Nessun elenco delle dipendenze in un file. In un computer di lavoro nuovo, gli strumenti si reinstallano a mano leggendo il README [4].

Medi: testi vecchi o doppi

12. Il README sbaglia sugli hook. Dice «oggi in modalità avviso, non attivi». Vale per l'hook del manoscritto, ma il recinto blocca [9].

13. `PROCEDURA.md` non descrive il recinto. La sezione 10 parla degli hook di sessione e del manoscritto, non del recinto né di `zb lavoro` [10].

14. L'archivio è una fotografia. `INFRASTRUTTURA_COMPLETA.md` copia i file al commit `668afc2`. Ogni modifica

successiva lo rende vecchio, e nessun controllo lo segnala [26].

15. Due esportazioni parallele in `docs/`. Ci sono anche `ESPORTAZIONE_COMPLETA_1…6.md`, un'altra copia del progetto, senza verifica di allineamento [27].

16. I comandi sono elencati tre volte. Nell'aiuto di `zb`, nel README e in `PROCEDURA.md`. Già oggi non coincidono: `zb lavoro`, `zb push`, `zb fase` mancano nella lista principale del README [23].

17. Istruzioni di LanguageTool due volte. Nel README e in `PROCEDURA.md`: vanno aggiornate insieme [22].

18. Arcaismi copiati in 10 file. Lo stesso elenco di arcaismi vietati è ripetuto nei divieti di molte voci. Una modifica va fatta in ogni file [20].

19. Lo stesso `materiali/LEGGIMI.md` in 11 voci. File identici, ripetuti [21].

20. Il modello di `libro.yaml` non mostra le appendici. Il campo c'è nello schema e nella procedura, non nel modello [15].

Lievi: dettagli

21. 14 o 19 scenari? `dati/hook.yaml` parla di «14 scenari»; la tabella di `PROCEDURA.md` §10 ne ha 19 [12].

22. Un commento sbagliato. `dati/struttura-libro.yaml` dice di essere usata da `pulizia.py`, che invece non la legge [13].

23. Un nome di ramo vecchio. `proposta-motore.md` §B.4 parla di un ramo `claude/motore` con richieste di unione; oggi il ramo è `motore` [14].

24. Numeri di cartella doppi. `05-revisioni`, `05-appendici`, `05-output` hanno lo stesso numero; così `06-diagnostica` e `06-pubblicazione`. Funziona, ma l'ordine alfabetico non è quello di lavoro [16].

25. Un ramo di sessione sbagliato. Il ramo assegnato alla sessione dell'archivio conteneva un altro progetto, non il motore [19].

26. Skill citate ma assenti. Vedi capitolo 6 [3].

Una scelta dichiarata, non un difetto

27. `CLAUDE.md` diversi nei rami dei libri vecchi. L'archivio lo elenca come punto debole. È invece una scelta dell'autore: i libri vecchi sono copie di lavoro per la riscrittura, con un `CLAUDE.md` proprio, da non riallineare (capitolo 5) [17].

(Tutto il capitolo: docs/INFRASTRUTTURA_COMPLETA.md §10, punti 1-27.)

9. Dove trovare le cose

Ramo «motore» vuol dire anche: nella copia del motore di ogni ramo di libro aggiornato.

Cosa cerchi	File	Ramo
Regole di avvio	`CLAUDE.md`	motore (ogni ramo di libro ha il suo)
Procedura completa	`motore/PROCEDURA.md`	motore
Elenco dei comandi	`motore/README.md`, «Comandi»; `python3 motore/zb aiuto`	motore
Stato del lavoro sul motore	`motore/passaggio-di-consegne.md`	motore
Limiti noti del motore	`motore/report-motore.md` §11	motore

Cosa cerchi	File	Ramo
Progetto originale	`motore/proposta-motore.md`	motore
Profili di genere	`motore/profili/`	motore
Modelli vuoti	`motore/modelli/`	motore
Modulo del briefing	`motore/modelli/briefing.md`	motore
Dati KDP	`motore/dati/kdp.yaml`	motore
Lista nera	`motore/dati/lista-nera.yaml`	motore
Parole italiane ammesse	`motore/dati/parole-ammesse-it.txt`	motore
Cartelle di un libro	`motore/dati/struttura-libro.yaml`	motore
Campi di `libro.yaml`	`motore/dati/libro.schema.yaml`	motore

Cosa cerchi	File	Ramo
Hook	`motore/script/hook_*.py`, `motore/dati/hook-settings.esempio.json`	motore
Modalità dell'hook del manoscritto	`motore/dati/hook.yaml`	motore
Hook accesi	`.claude/settings.json`	ramo del libro
Impaginazione e font	`motore/stampa/`	motore
Prove e mini-libri inventati	`motore/script/prove.py`, `motore/prove/`	motore
Libro attivo	`libri/libro-attivo.md`	ramo del libro
Un libro	`libri/<slug>/`	ramo del libro

Cosa cerchi	File	Ramo
Briefing del libro	`libri/<slug>/00-progetto/briefing.md`	ramo del libro
Bibbia del libro	`libri/<slug>/02-bibbia/bibbia.md`	ramo del libro
Capitoli	`libri/<slug>/04-manoscritto/`	ramo del libro
Appendici	`libri/<slug>/05-appendici/`	ramo del libro
PDF e libro compilato	`libri/<slug>/05-output/`	ramo del libro
Report dei controlli	`libri/<slug>/06-diagnostica/`	ramo del libro
Pacchetto KDP	`libri/<slug>/06-pubblicazione/`	ramo del libro
Report di sessione del libro	`libri/<slug>/passaggio-di-consegne.md`	ramo del libro

Cosa cerchi	File	Ramo
Voci narranti	`indice.yaml`, `voci/<slug>/`	banca-voci, ramo `main` (repository a parte)
Archivio completo	`docs/INFRASTRUTTURA_COMPLETA.md`	motore
Questa guida	`docs/GUIDA_AL_SISTEMA.pdf` e `.md`	motore

(docs/INFRASTRUTTURA_COMPLETA.md §8.4 e §9; motore/dati/struttura-libro.yaml; motore/README.md)

Fine della guida.